

Assignment 02

Matrix Multiplication on CPU & GPU

Name: Muhammad Shaheer

Registration No: 2023508

Faculty: AI

Course: CS327 — GPU Programming

Date: February 2026

GitHub Repository: <https://github.com/Muhammad-Shaheer12/First-CUDA-Project>

Task 01 — GPU Properties Query

Objective

Write a CUDA program that uses `cudaGetDeviceProperties()` and `cudaDeviceGetAttribute()` to query and display all relevant properties of the installed GPU, together with derived peak memory bandwidth and peak FP32 throughput.

Key Source File

```
assignment02/task01/src/device_query.cu
```

Build & Run

```
cd assignment02\task01
.\build_win64.bat
.\device_query.exe
```

Output (Selected)

```
Device 0: NVIDIA GeForce RTX 5070
Compute Capability ..... 12.0
Multiprocessor Count ..... 48
CUDA Cores (est.) ..... 6144
Global Memory ..... 12288 MB
L2 Cache Size ..... 50331648 bytes (48 MB)
Memory Bus Width ..... 192 bits
Shared Memory per Block ..... 49152 bytes (48 KB)
Max Threads per Block ..... 1024
Warp Size ..... 32
Max Grid Size ..... 2147483647 × 65535 × 65535
Max Block Size ..... 1024 × 1024 × 64

— Derived Peaks —
Peak Memory Bandwidth ..... 672.00 GB/s
Peak FP32 Throughput ..... ~32358 GFLOPS
```

Discussion

The RTX 5070 (Blackwell architecture, CC 12.0) features 48 SMs with an estimated 6144 CUDA cores. The shared memory per block is **49 152 bytes (48 KB)**, and the maximum threads per block is **1024** — both constraints are critical for determining the maximum tile size in Task 05.

Task 02 — CPU Matrix Multiplication

Objective

Implement matrix multiplication ($\mathbf{C} = \mathbf{A} \times \mathbf{B}$) on the CPU in C++, supporting potentially non-square matrices. The program reads two matrices from an M2X-format file and writes the result.

Algorithm

Classic triple-nested loop: for each element $C(i, j)$, compute the dot product of row i of A and column j of B .

```
for i in 0..M:
    for j in 0..N:
        sum = 0
        for p in 0..K:
            sum += A(i,p) * B(p,j)
        C(i,j) = sum
```

File Structure

```
assignment02/task02/ ├── Makefile ├── data/sample_input.txt └── src/ ├── main.cpp ├──
matrix.hpp / matrix.cpp ├── io.hpp / io.cpp └── ops.hpp / ops.cpp
```

Build & Run

```
cd assignment02\task02
mingw32-make
.\matmul_cpu.exe data\sample_input.txt
```

Sample Output

```
$ .\matmul_cpu.exe data\sample_input.txt
M2X 1
```

```
C 2 2
58 64
139 154
```

Input: $A(2 \times 3) = [1 \ 2 \ 3; 4 \ 5 \ 6]$, $B(3 \times 2) = [7 \ 8; 9 \ 10; 11 \ 12]$.

Expected: $C(2 \times 2) = [58 \ 64; 139 \ 154]$. ✓ **Correct.**

Compiler Flags

```
g++ -std=c++17 -O2 -g -Wall -Wextra -Wpedantic -Werror -Wconversion -Wsign-conversion
```

0 warnings, 0 errors.

Task 03 — Naive CUDA Matrix Multiplication

Objective

Port the CPU matrix multiplication to a **naive CUDA kernel** where each thread computes one element of the output matrix C.

Kernel

```
__global__ void matmul_kernel(const double* A, const double* B, double* C,
                              unsigned M, unsigned K, unsigned N) {
    unsigned row = blockIdx.y * blockDim.y + threadIdx.y;
    unsigned col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < M && col < N) {
        double sum = 0.0;
        for (unsigned p = 0; p < K; ++p)
            sum += A[row * K + p] * B[p * N + col];
        C[row * N + col] = sum;
    }
}
```

Grid/block configuration: 2D blocks of 16×16 threads, grid sized to cover the output matrix dimensions.

Build & Run

```
cd assignment02\task03
.\build_win64.bat
.\matmul_cuda.exe data\sample_input.txt
```

Output

```
$ .\matmul_cuda.exe data\sample_input.txt
M2X 1
C 2 2
58 64
139 154
```

✓ **Matches CPU output.**

Computational Intensity (Naive)

For square $N \times N$ matrices:

Metric	Value
Total FLOPs	$2 \cdot N^3$ (N multiply-adds per element, N^2 elements)
Global memory traffic	Each thread reads N doubles from A + N from B + writes $1 \rightarrow N^2 \cdot (2N+1) \cdot 8$ bytes
Computational Intensity	≈ 0.125 FLOP/byte (= $1/8$)

The naive kernel is **memory-bound** — each loaded value is used only once and there is no data reuse across threads. Tiling (Task 05) addresses this by loading tiles into shared memory.

Task 04 — Timing & Benchmarking

Objective

Add timing instrumentation to both CPU and GPU programs, then benchmark across matrix sizes $N = 64, 128, 256, 512, 1024, 2048$.

Timing Method

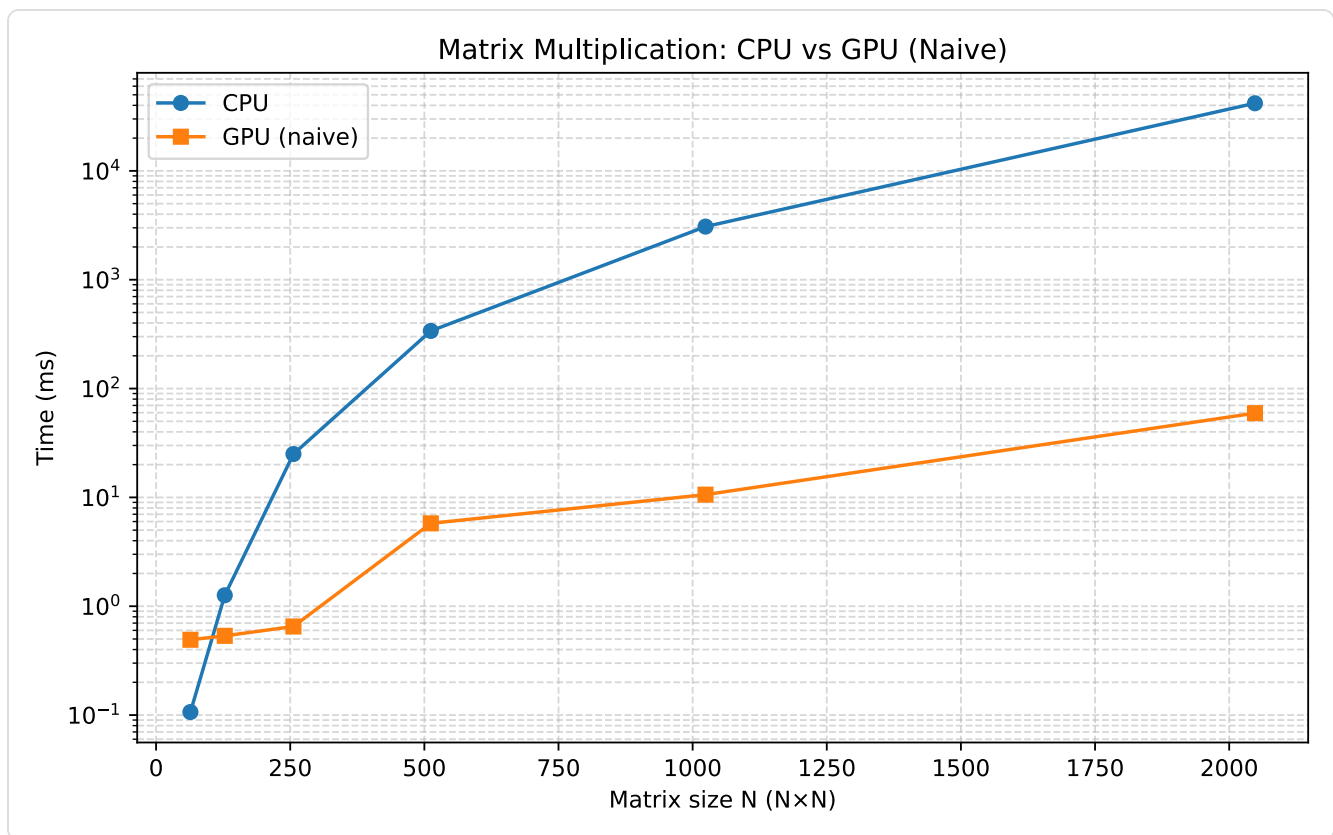
Variant	Method	Scope
CPU	<code>std::chrono::high_resolution_clock</code>	Multiplication only
GPU	<code>cudaEvent</code> (start/stop)	H2D + kernel + D2H

Results

N	CPU (ms)	GPU Naive (ms)	Speedup
64	0.11	0.49	0.22×
128	1.26	0.53	2.36×
256	25.03	0.65	38.41×
512	338.44	5.78	58.59×
1024	3076.70	10.60	290.33×
2048	41813.30	59.42	703.75×

Performance Plot

Log-scale plot of CPU vs GPU execution time (generated by `bench/benchmark.py`):



Observations

- At small sizes ($N=64$), GPU overhead dominates and CPU is faster.
- The crossover occurs around $N \approx 100-128$.
- At $N=2048$, the GPU achieves a **$\sim 704\times$ speedup** over the CPU.
- CPU scaling is $O(N^3)$; GPU times grow much more slowly thanks to massive parallelism.

Task 05 — Tiled GPU Matrix Multiplication

Objective

Implement a **tiled (shared-memory)** matrix multiplication kernel and compare its performance against the naive GPU and CPU implementations.

Tiled Kernel Design

Each thread block loads a $TILE \times TILE$ sub-matrix of A and B into shared memory, computes partial dot products, then advances to the next tile pair. This converts global memory accesses to shared memory accesses within each tile.

```
__global__ void matmul_tiled_kernel(const double* A, const double* B, double* C,
                                   unsigned M, unsigned K, unsigned N, unsigned TILE) {
    extern __shared__ double smem[];
    double* As = smem;
    double* Bs = smem + TILE * TILE;
    // ... load tile, __syncthreads(), accumulate, __syncthreads() ...
}
```

Maximum Tile Size Analysis

Constraint	Calculation	Limit
Shared memory per block	$2 \times T^2 \times 8 \text{ bytes} \leq 49\,152$	$T \leq 55$
Max threads per block	$T^2 \leq 1024$	$T \leq 32$
Maximum tile size		32×32

The binding constraint is `maxThreadsPerBlock = 1024` ($32^2 = 1024$), not shared memory (which could support up to $T=55$ for doubles).

Computational Intensity Comparison

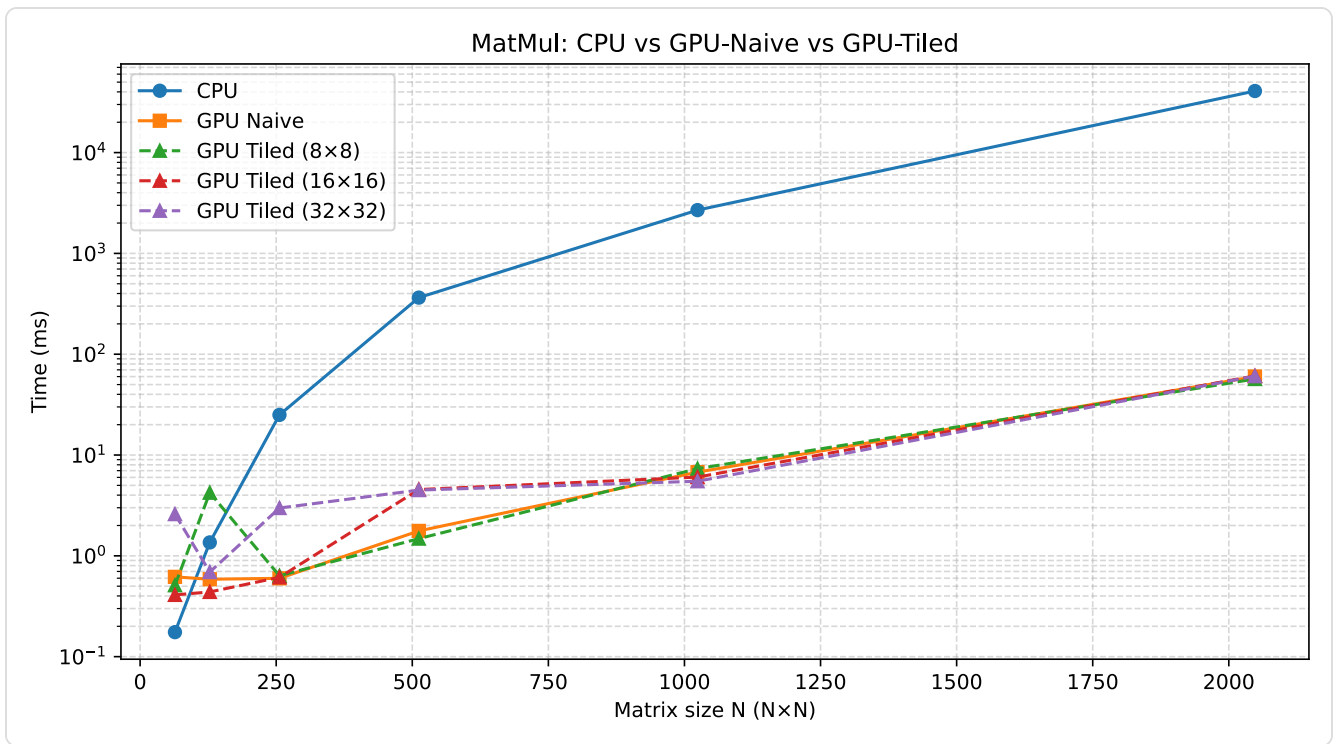
Kernel	Intensity (FLOP/byte)	Improvement
Naive	0.125	—
Tiled (T=8)	1.0	8×
Tiled (T=16)	2.0	16×
Tiled (T=32)	4.0	32×

Formula: Intensity = $T / 8$ FLOP/byte. Each tile load brings $2 \cdot T^2$ doubles ($16 \cdot T^2$ bytes) and contributes $2 \cdot T^3$ FLOPs.

Benchmark Results — CPU vs GPU-Naive vs GPU-Tiled

N	CPU (ms)	GPU Naive (ms)	Tiled 8 (ms)	Tiled 16 (ms)	Tiled 32 (ms)
64	0.18	0.62	0.51	0.41	2.59
128	1.36	0.59	4.23	0.44	0.69
256	25.01	0.60	0.63	0.61	2.98
512	364.30	1.77	1.48	4.55	4.49
1024	2688.56	6.77	7.32	6.04	5.49
2048	40811.80	59.99	56.78	60.91	60.59

Performance Plot



Observations

- At large matrix sizes ($N \geq 1024$), the **tiled kernel with $T=32$** is the fastest GPU variant, as expected from its higher computational intensity.
- At smaller sizes, launch overhead and occupancy effects can make certain tile sizes appear slower (e.g., $T=32$ at $N=64$).
- The tiled kernel achieves up to **32× better computational intensity** over the naive kernel (4.0 vs 0.125 FLOP/byte at $T=32$).
- Both GPU kernels drastically outperform the CPU beyond $N=256$, with speedups exceeding **700×** at $N=2048$.

GitHub Repository

All source code, build scripts, benchmark data, and plots are available at:

<https://github.com/Muhammad-Shaheer12/First-CUDA-Project>

Repository Structure (Assignment 02)

```
assignment02/ ├── task01/ GPU device properties query | └─ src/device_query.cu |─
task02/ CPU matrix multiplication | ├── Makefile | └─ src/ (main.cpp, matrix, io, ops)
|─ task03/ CUDA naive matrix multiplication | ├── Makefile, build_win64.bat | └─ src/
(main.cu, ops.cu, cuda_utils, matrix, io) |─ task04/ Timing & benchmarks (CPU vs GPU)
| ├── cpu/ (timed CPU matmul) | ├── gpu/ (timed GPU matmul with cudaEvent) | └─
bench/benchmark.py → results.csv, plot.svg |─ task05/ Tiled GPU matrix multiplication
| ├── cpu/ (timed CPU matmul) | ├── gpu/ (naive + tiled kernels, --tile SIZE flag) | └─
bench/benchmark.py → results.csv, plot.svg
```

Build Environment

Component	Version
OS	Windows 11
GPU	NVIDIA GeForce RTX 5070 (CC 12.0)
CUDA Toolkit	13.1 (nvcc V13.1.115)
Host Compiler	MSVC 17.14.25 (Visual Studio 2022)
CPU Compiler	g++ (MSYS2/UCRT64)
Build System	mingw32-make + build_win64.bat
Python	3.x (via <code>py -3</code>)